

# CS 415 Operating Systems Spring 2026

## Project #2 Report Collection

Author:

*Lucas Lee Bixby*

*UO ID: lbixby*

*952001359*

# Report

## Introduction:

*In this project, I was tasked with building a simple MCP ( Master Control Program ) used to extract a workload of commands from an input source text file, launch the processes to run concurrently, implement scheduling behavior, and display process information at each cycle. This project came together in four different parts, each new step building off the previous. Part 1 extracted the commands and launched the workload. Part 2 allowed for the MCP to control the flow of the processes using signals ( SIGSTOP/ SIGCONT ). Part 3 implemented the scheduling algorithm ( Round-Robin ). And finally, Part 4 displayed a table with information on each process.*

*This project was important in understanding how the OS handles programs. I learned about child processes/forking, signals/ system calls, scheduling algorithms, and process data.*

## Background:

*First off, I was consistent in my project architecture, which played a vital role in maintaining organization and consistency as I scaled my project to add more functionality. I managed this task by creating a helper file named helpers.c, which contained all my helper functions, and the header file MCP.h, which linked each function to its part of the project. On top of this, I also created a GitHub repository to manage the version control and commit history of my project, for organization, and practice with professional industry tools. These combined methods made scaling my code efficient and clean. When I needed to add the next part of my project, I would copy over the previous code and edit the handle\_workload() function to reference or adapt to the newly defined part-specific functions in helpers.c. Additionally, to add another layer of organizational methods, after working on the project for a while, I would write myself development notes as a comment in the header of the code I was working on, to preserve my train of thought for when I returned.*

*Next, another algorithm/ method that I utilized in my implementation of this project was my technique for parsing the command lines as a workload before executing. This technique was adopted from my code in Project\_1, where I used the functions: count\_lines(), parse\_args(), extract\_commands(), and free\_workload() to format the input file into an array of command\_line structs, which were then fed into the handle\_workload function for strategic execution.*

*Another algorithm/ method I used in this project for the MCP scheduling was a simple Round-Robin algorithm. For this, I needed to create enqueue and dequeue functions, along with an alarm\_handler() for the waitpid(). The strategy of this algorithm was to: enqueue each process as it was launched, stop all processes after they had all started, then execute the process at the beginning of the queue for one second. Then, stop the process and check to see whether it has completed or still has work to do. If the process had completed, the scheduler would dequeue it and decrement the active counter. If the process was not finished, then the scheduler would dequeue the process and immediately enqueue it, effectively sending it to the back. This allows for each process to get an equal share of the CPU until all processes have finished. This method provides an equal share of the work, but does not contain any smart logic based on the requirements and needs for each process. A more complex scheduling algorithm would take these factors into account before deciding on which process to execute next.*

*Finally, the last unique method that I chose to use was creating setup\_signals() and wait\_for\_signal() helper functions. This effectively allowed me to create a signal set, including the relevant signals, such that I could block those signals with sigprocmask(), so that sigwait() intercepts them, and allowed me to implement*

my own scheduling behavior instead of letting the OS apply the default signal behavior. This allowed me to coordinate execution, handle manual time slicing with signals, and build a process monitoring layer using /proc.

## Implementation:

One instance to demonstrate my implementation strategy was in the enqueueing and dequeuing of the Round-Robin algorithm. Because I knew how many processes were initially launched, I set my queue to be that length, then I stored the integer value of the launched processes in a variable named active, which was decremented every time a process finished. This allowed me to use the “active” counter to gauge the end of the queue for future unfinished process enqueueing. Additionally, my dequeue method rotated all of the process IDs such that the next process was always at the front of the queue.

```

190
191 void enqueue(pid_t *pids, int active, pid_t id)
192 {
193     pids[active] = id;
194 }
195
196 pid_t dequeue(pid_t *pids, int active)
197 {
198     pid_t removed = pids[0];
199     for ( int i=0; i < active - 1; i++) {
200         pids[i] = pids[i+1];
201     }
202     pids[active - 1] = -1;
203
204     return removed;
205 }
206

```

The following code snippet represents the conditional logic for determining how to handle the process after a process time slice completes, by interpreting the return value of waitpid().

```

224
225     int status;
226     pid_t result = waitpid(current, &status, WNOHANG);
227
228     if ( result == 0 ) {
229         // process still active -> put back into the queue
230         enqueue(pids, active - 1, current);
231     }
232     else if ( result == current ) {
233         // process finished
234         active--;
235     } else {
236         perror("waitpid");
237         active--;
238     }
239

```

## Performance Results and Discussion

*The performance of my project is up to par in terms of runtime, memory allocation, and output. I developed my assignment for Project\_2 through an SSH connection to the University of Oregon's ix-dev system, and there were no noticeable performance issues when running tests or the final program. Although I didn't have the time to optimize my code to peak its performance, my provided architecture matches the assignment details and specifications. As for memory allocation and clean up, I ran my program with Valgrind to check for memory leaks; in all cases for parts 1-4, they pass (valgrind logs 1-4.txt are contained in the Project 2 .zip file).*

*Here is a screenshot of my program running on the ix-dev system against the provided test\_script.sh :*

```

● lbixby@ix-dev:~/lucas_bixby_homework/lucas_bixby_assignments/CS415/project_2$ ./test_script.sh
rm -f *.o part1 part2 part3 part4 iobound cpubound
gcc -Wall -g -c part1.c -o part1.o
gcc -Wall -g -c helpers.c -o helpers.o
gcc -Wall -g part1.o helpers.o -o part1
gcc -Wall -g -c part2.c -o part2.o
gcc -Wall -g part2.o helpers.o -o part2
gcc -Wall -g -c part3.c -o part3.o
gcc -Wall -g part3.o helpers.o -o part3
gcc -Wall -g -c part4.c -o part4.o
gcc -Wall -g part4.o helpers.o -o part4
gcc -Wall -g iobound.c -o iobound
gcc -Wall -g cpubound.c -o cpubound

== Testing if part1 takes the correct arguments... ==
Success: 'part1' is taking the -f flag correctly

== Testing if part1 has correct output... ==
Success: part1 took the right amount of time
Success: 'part1' has the correct output

== Testing if part2 has correct output... ==
Success: part2 took the right amount of time
Success: 'part2' has the correct output

== Testing if part3 has correct output... ==
Success: part3 took the right amount of time
Success: 'part3' has the correct output

== Testing if part4 has correct output... ==
Success: part4 took the right amount of time
Success: Table detected

```

## Conclusion

*Overall, this project was really interesting in how we utilized signals and system calls to manage the flow of processes. I definitely learned a lot of useful skills and techniques that gave me a deeper understanding of how operating systems work. I think that if I were to come back and revisit this project in the future. I would implement a smart scheduler that decides on the next process to run based on the needs of the process, against other processes in the waiting queue. This would follow a more complex and precise scheduling algorithm that would ideally be more efficient than the current Round-Robin.*